MERGE SORT – MEMORY UTILIZATION ANALYSIS

1. Aim

To analyze the memory utilization of Merge Sort by tracking its auxiliary space usage during execution for different input sizes, and to study the relationship between additional space, stability, time complexity, and space complexity.

2. Algorithm

1. Start with an unsorted array of n elements.
2. Divide the array into two halves.
3. Recursively apply Merge Sort to the left half.
4. Recursively apply Merge Sort to the right half.
5. Create a temporary array for merging the two sorted halves.
6. Compare elements from both halves.
7. Copy the smaller element into the temporary array.
8. Continue until all elements are merged.
9. Copy the elements from the temporary array back to the original array.
10. Repeat until the complete array is sorted.
11. Track the auxiliary memory used during execution.
12. Display the sorted array and memory observations.

3. Pseudocode

```
MERGE_SORT(A, low, high)

   if low < high then
      mid = (low + high) / 2

      MERGE_SORT(A, low, mid)
      MERGE_SORT(A, mid + 1, high)

      MERGE(A, low, mid, high)
   end if

END MERGE_SORT

MERGE(A, low, mid, high)

   Create temporary array TEMP
```

```
    i = low
    j = mid + 1
    k = 0

    while i <= mid AND j <= high do
        if A[i] <= A[j] then
            TEMP[k] = A[i]
            i = i + 1
        else
            TEMP[k] = A[j]
            j = j + 1
        end if
        k = k + 1
    end while

    while i <= mid do
        TEMP[k] = A[i]
        i = i + 1
        k = k + 1
    end while

    while j <= high do
        TEMP[k] = A[j]
        j = j + 1
        k = k + 1
    end while

    Copy TEMP into A

END MERGE
```

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

int maxSpace = 0;

void mergeSort(int a[], int l, int r) {
    if (l >= r) return;

    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);

    int n = r - l + 1;
    int *temp = malloc(n * sizeof(int));

    if (n > maxSpace)
        maxSpace = n;

    int i = l, j = m + 1, k = 0;

    while (i <= m && j <= r)
        temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];

    while (i <= m) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];

    for (i = 0; i < n; i++)
        a[l + i] = temp[i];

    free(temp);
}

int main() {
    int n;

    printf("Enter number of elements: ");
```

```c
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *a = malloc(n * sizeof(int));

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nMaximum auxiliary space: %d elements", maxSpace);
    printf("\nMemory used: %d bytes", maxSpace * (int)sizeof(int));

    free(a);
    return 0;
}
```

Sample Output

```
Enter number of elements: 7
Enter elements: 1 3 5 7 9 2 4
Sorted array: 1 2 3 4 5 7 9
Maximum auxiliary space: 7 elements
Memory used: 28 bytes
```

6. Memory Consumption Analysis

Merge Sort primarily uses a temporary array during the merge operation. For an input containing n elements, the temporary array requires approximately O(n) auxiliary space. The recursive calls require O(log n) stack space.

Auxiliary Space = O(n) + O(log n) = O(n)

The exact memory measured by a program may vary depending on the compiler, operating system, and implementation. The theoretical growth remains O(n).

Input Size	Auxiliary Space	Observation
10	O(10)	Low
100	O(100)	Moderate

500	O(500)	Higher
1000	O(1000)	High
5000	O(5000)	Very High
10000	O(10000)	Very High

7. Stability Analysis

Merge Sort is a stable sorting algorithm when the merge operation is implemented appropriately. During merging, when two elements have equal values, the element from the left subarray is selected first using the condition A[i] <= A[j]. This preserves the original relative order of equal elements.

For example, before sorting: (50,A) (30,B) (50,C) (20,D). After stable sorting: (20,D) (30,B) (50,A) (50,C). The two elements with value 50 remain in their original order.

The temporary array makes the merging process easier to perform without overwriting elements that are still required. Thus, the additional memory supports the stable implementation of Merge Sort.

8. Time Complexity

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Merge Sort has the same asymptotic time complexity in all three cases because the array is repeatedly divided and the resulting subarrays are merged.

9. Space Complexity

Temporary Array: O(n)

Recursion Stack: O(log n)

Overall Auxiliary Space: O(n)

The temporary array is the major contributor to memory usage.

10. Time-Space Trade-off

Merge Sort uses additional memory to achieve efficient and predictable sorting.

- Maintains O(n log n) worst-case time complexity.
- Supports stable sorting.

- Makes the merging operation easier.
- Provides predictable performance for large inputs.

The main disadvantage is that Merge Sort requires O(n) additional memory, and memory consumption increases as the input size increases.

Therefore, Merge Sort makes a trade-off: more memory is used in exchange for stable and predictable O(n log n) sorting. An algorithm such as Heap Sort uses O(1) auxiliary space but does not provide stable sorting. Thus, Merge Sort is preferable when stability is important.

11. Result

Merge Sort was analyzed for different input sizes. The analysis shows that its auxiliary memory usage increases approximately linearly with the input size.

Time Complexity: O(n log n)

Auxiliary Space: O(n)

Recursion Space: O(log n)

Stability: Stable

The additional memory required by Merge Sort provides stable merging and guaranteed O(n log n) time performance.